



DBDOME

Cross-Vendor Data Migration Engine

Source server + database set -> JSON ruleset in Postgres -> destination server + database set
Status: design, for approval | Target first source vendor: IBM DB2

This plan describes an end-to-end migration solution that extracts from a source server's set of databases, transforms each row against rules stored as JSON in Postgres tables, and loads the result into a destination server's set of databases - on a schedule. It is deliberately built from parts DBDOME already runs: the server registry and its encrypted credentials, the JSON-rule execution pattern used by the detection tree, the column-rule cache used by data masking, the interval scheduler, and the run-accounting shape introduced with the mail diagnostics. The new work is the rule schema, the extract-transform-load engine, type mapping and reconciliation.

~70%	of the plumbing already exists in this codebase
DB2	driver verified on the exact build interpreter
8 + 3	new Postgres tables and operator views (written, validated)
3-4 wk	to a production-quality first vendor pair

Contents

VERDICT	Feasibility and the DB2 question
SHAPE	The solution in one picture
MODEL	The Postgres data model
RULES	The transformation ruleset
SCHED	Scheduling
ENGINE	The engine
PHASES	Delivery plan
RISK	Risks and decisions outstanding
REUSE	What already exists

Generated 17 Sep 2026 | DDL: `sql_scripts/7680_migration_engine.sql`



Feasibility and the DB2 question (VERDICT)

What was actually tested on this machine, and what it costs.

YES

The solution is feasible, and most of it is assembly rather than invention

DBDOME already connects to seven vendors with encrypted credentials, already stores executable rules as JSON in Postgres and runs them against a target connection, already schedules recurring work with a live enable/disable toggle, and already has a run-accounting pattern that records the stage a failure happened at. The migration engine reuses all five.

VERIFIED

DB2 as a source works on the exact interpreter the build uses

ibm_db 3.3.0 publishes a cp314 wheel; it was downloaded and imported successfully on Python 3.14.5 on this machine. The first import failed with 'DLL load failed' and succeeded once clidriver/bin was registered with `os.add_dll_directory()` - the same pattern `utils/oracle_client.py` already uses for the Oracle Instant Client. So the integration shape is one this codebase has solved before.

COST

About 86 MB added to the bundle, plus a spec entry and a runtime hook

Almost all of it is IBM's bundled CLI driver. The alternative is `pyodbc`, which is already in the build, against the IBM Data Server ODBC driver - no new Python dependency, but the driver must then be installed on every host.

CAVEAT

The risk is which DB2, not whether DB2

LUW, DB2 for i (AS/400) and z/OS differ in catalog views (`SYSCAT.*` vs `QSYS2.*` vs `SYSIBM.*`), paging syntax, and - on i and z - CCSID/EBCDIC encoding plus `GRAPHIC` and `VARGRAPHIC` types. The engine is unaffected; the introspection SQL and type map are per-flavour. Decide the flavour before Phase 1.

There is no DB2 instance on this network to develop against. DB2 Community Edition in Docker on the Ubuntu VM is the cheapest way to unblock Phase 1, and should be stood up before any engine work starts.



The solution in one picture (SHAPE)

Four moving parts; the middle two are rows in Postgres, not code.

Flow

```
source server (metrics.servers.server_id)
+-- database set      <- migration.database_map (source_database)
    |
    v
transformation ruleset <- migration.rules (JSONB)
  project -> database -> object -> column, merged, most specific wins
    |
    v
destination server (metrics.servers.server_id)
+-- database set      <- migration.database_map (target_database)

driven by: metrics.registered_processes 'migration_runner' (60s tick)
          + migration.schedules (per-project cadence and guardrails)
```

WHY

Connection identity is never duplicated

An endpoint is a reference to metrics.servers.server_id, so host, port, vendor, driver and the enc:v1: credentials keep living where the existing server form manages them. Because metrics.servers.database is a single column, the database SET cannot live there - it is migration.database_map.

SETS

One table makes renames, subsets and consolidation the same thing

A 1:1 rename is one row. A subset is fewer rows. Many-to-one consolidation is several rows sharing a target_database. No code path distinguishes them.



The Postgres data model (MODEL)

Written as `sql_scripts/7680_migration_engine.sql` and validated against the live database inside a transaction that was rolled back.

Tables

migration.projects	Source and target server_id, mode (full/incremental), object_scope, batch size, parallelism, on_error policy, and allow_write - the dry-run gate, false by default.
migration.database_map	The database set on both sides: source_database -> target_database, schema_map JSONB, load_order, is_active.
migration.rules	The JSON ruleset. scope in (project, database, object, column) with a CHECK that each scope carries exactly the keys it needs. GIN-indexed.
migration.schedules	Per-project cadence plus the guardrails a long destructive job needs. One row per project.
migration.runs	One row per execution: trigger, dry_run, status, counts, heartbeat, host.
migration.run_items	One row per (database, object): status, STAGE, rows read/written/rejected, duration, error class and code.
migration.watermarks	Per (project, database, object) resume point - restartability and incremental loads.
migration.rejects	Quarantined rows with the rule that rejected them, so bad data never silently disappears.

Operator views

migration.v_schedule_due	Why each project will or will not start on this tick.
migration.v_run_errors	Every failed or skipped object, newest first.
migration.v_run_summary	Per-run totals: objects ok/failed, rows, duration.

Two defaults are deliberately conservative: allow_write starts false, so a new project can only dry-run until someone explicitly enables writing; and object_scope starts 'listed' (opt-in per object) rather than 'all'. Both are one-column changes if the opposite posture is preferred.



The transformation ruleset (RULES)

Rules are rows. Changing a mapping is an UPDATE, never a rebuild - the same property that lets the Oracle tester's query file be swapped in bin.

MERGE

Resolution is project -> database -> object -> column

Rules are merged rather than replaced, so a project-level default such as 'trim all text' or 'DECIMAL maps to numeric' applies everywhere while an object or column rule overrides only what it names. This mirrors config.masking_rules keyed by (server, table, column) and cached by processes/data_masking_engine.py, one level deeper.

Project scope - the defaults everything inherits

```
{ "type_map": { "DECIMAL": "numeric", "DECFLOAT": "numeric",
               "VARGRAPHIC": "text", "TIMESTAMP": "timestamp(6)",
               "CLOB": "text", "BLOB": "bytea" },
  "defaults": { "trim_text": true, "empty_string_as_null": true,
               "batch_size": 5000 },
  "exclude": { "objects": ["*_TMP", "SYS*"] } }
```

Object scope - one table

```
{ "source": { "select": "SELECT * FROM DB2INST1.CUSTOMERS
                    WHERE UPD_TS > :watermark",
              "params": { "watermark": { "from": "watermark",
                                         "type": "timestamp" } },
              "order_by": ["CUST_ID"] },
  "target": { "table": "public.customers", "load": "upsert",
              "keys": ["cust_id"],
              "post_sql": ["ANALYZE public.customers"] },
  "validate": { "row_count": "equal", "checksum_keys": ["cust_id"] } }
```

Column scope - overrides only what it names

```
{ "src": "SSN", "dst": "ssn", "type": "text",
  "transform": [ { "op": "trim" },
                 { "op": "mask", "rule": "ssn_last4" } ] }
```

SAFETY

Transforms are a whitelisted operator registry, never eval

trim, cast, nullif, concat, lookup, date_parse, mask and a restricted expr subset. A rule stored in the database must never be able to execute arbitrary Python: the rules table is operator-editable, so it is an injection surface by definition.

SEED

Rules are drafted, not hand-written from nothing

monitoring.schema already holds 2,896 discovered table/column/data_type rows. The generator drafts column rules from it against the project type map, and the operator edits only the exceptions. For a DB2 source not yet inventoried, the connector reads SYSCAT.COLUMNS.



Scheduling (SCHED)

Two layers, both already running in this product. No third scheduler is introduced.

TICK

One new registered process: migration_runner, 60s, seeded INACTIVE

metrics.registered_processes already drives interval jobs, resolves a process name to a callable via get_function_by_name, and is re-read by the config watcher every 30 seconds - so a migration dispatcher can be enabled or disabled live, with no restart. It is seeded inactive on purpose: creating tables is what makes the feature possible, starting a loop that can write to customer databases is a separate, deliberate act.

CADENCE

Per-project schedule rows, modelled on jobs.job_schedules

schedule_kind is interval, daily, weekly, monthly, cron or manual, with a CHECK constraint that the chosen kind brings its own parameters - a daily schedule without a time is rejected by the database, not by hope.

Guardrails a report schedule never needed

Single-flight	state + heartbeat_at + lock_key (generated, for pg_try_advisory_lock). A tick must never start a project that is already running, including from a second service instance.
Crash recovery	stale_after_seconds. A service killed mid-run would otherwise leave state='running' forever; a stale run becomes dispatchable again.
Maintenance window	allow_from / allow_to / allow_days, plus kill_outside_window. Big loads run at night.
Retry with backoff	attempt, max_attempts, retry_backoff_seconds. A dead source is backed off, not hammered every tick.
Catch-up policy	run_once or skip. After downtime a report may replay; a migration must never replay every missed slot.
Runtime cap	max_runtime_seconds, priority, consecutive_failures.

Verified dispatcher decisions (v_schedule_due, exercised in a rolled-back transaction)

due now	-> due	stale=False
not due yet	-> not due yet	
already running	-> already running	
stale -> recoverable	-> due	stale=True
schedule inactive	-> schedule inactive	
paused	-> paused	

WHY

A skipped migration must never be a mystery

v_schedule_due returns a dispatch_decision for every project, so 'it did not run last night' is answered by one query instead of by reading scheduler code. This is the same principle as the mail send log added earlier: record the reason, not just the outcome.



The engine (ENGINE)

Vendor-agnostic by construction: the only vendor-specific parts are the connector and the introspection SQL.

Loop

```
project
+- for each active database_map row      (up to parallel_databases)
+- resolve ruleset for that database
+- enumerate objects (rules union source catalog)
+- per object:
    extract batch (server-side cursor, batch_size)
    transform     (rule pipeline, rejects quarantined)
    load          (execute_values / COPY for PG targets)
    checkpoint    (migration.watermarks)
    validate      (row count, key checksums)
+- write migration.run_items: status, stage, counts, ms, error
```

RESTART

Checkpoint per batch, not per table

A killed run resumes at the watermark rather than restarting the object. The cost is that a failure can leave a partially loaded table; that is recoverable by design but must be a stated choice, not a surprise.

DRYRUN

Preview before any write exists as a first-class mode

`allow_write=false` extracts and transforms and writes nothing, reporting what would have been loaded. The operator page reuses the `/tester` pattern built for the Oracle root-cause tester: background run, live progress, stop button, PDF report at the end.



Delivery plan (PHASES)

Roughly three to four weeks to a production-quality first vendor pair; materially less for each additional source, because only the connector changes.

PHASE 1 (2-3 d)

DB2 connector and catalog introspection

utils/db2_client.py mirroring oracle_client.py's DLL-directory initialisation, the type-map table, and introspection proven against a real DB2 instance (Docker DB2 Community Edition on the Ubuntu VM). Gate: connect, list schemas, read a table, map every column type.

PHASE 2 (2-3 d)

Schema and rule model

Apply 7680_migration_engine.sql, add the JSON schema validator, and build the rule generator that drafts rules from monitoring.schema and from source introspection.

PHASE 3 (5-8 d)

The engine

Extract, transform, load, batching, checkpoints, rejects and run accounting. Gate: a full load of a non-trivial database with reconciliation clean.

PHASE 4 (2-3 d)

Operator page and dispatcher

/migration cloned from /tester, plus the migration_runner dispatcher consuming v_schedule_due and taking the advisory lock.

PHASE 5 (2-3 d)

Validation and reconciliation

Row counts, per-key checksums, sampled column comparison, and a PDF/CSV reconciliation report per run.

PHASE 6 (3-5 d, optional)

Incremental and CDC

Watermark-based incremental is included in the engine. True log-based CDC on DB2 LUW has no cheap equivalent of logical decoding and means IBM InfoSphere CDC or a third-party tool - set that expectation before it is promised to anyone.



Risks and decisions outstanding (RISK)

The ones that change the design if answered late.

Risks

Type fidelity	DECFLOAT, GRAPHIC/VARGRAPHIC, LOBs, and timestamp precision - DB2 offers 12 fractional digits where Postgres stops at 6. Needs an explicit, testable type map rather than per-rule guesswork.
Encoding	CCSID and EBCDIC on DB2 for i and z/OS.
Identity and keys	Sequence and identity resync after load, and FK-safe load ordering (load_order in database_map).
Throughput	execute_values is comfortable to about a million rows per object; beyond that use COPY with a binary writer.
Transactional boundary	Per-batch commit means a visible partial table during a run.
No test instance	There is no DB2 on this network today. Phase 1 is blocked until one exists.

Decisions needed

Which DB2 flavour	LUW, for i, or z/OS - drives introspection SQL, paging and encoding.
Object scope default	Opt-in per object ('listed', the safer default as written) or all-with-exclusions ('all', faster to set up, can pull staging tables into production).
Driver strategy	Bundle ibm_db (+86 MB, self-contained) or require the IBM ODBC driver on each host via the pyodbc already in the build.
Target write mode	Insert-only, upsert on keys, or truncate-and-load per object - it is per-rule, but a project default should be agreed.

Housekeeping to fold into Phase 3: utils/transformer_mssql_pg.py is the 49-line ancestor of this engine and carries a live production password in the repository. It should be deleted or gutted when the engine lands, and that credential rotated.



What already exists (REUSE)

Every row here is an existing, working part of this product - the reason the estimate is weeks rather than months.

Reuse map

Vendor connections, encrypted credentials	metrics.servers (17 servers, 7 vendors) + utils/secrets_crypto + per-vendor collectors, including Informix - the IBM-family precedent.
JSON rules in PG executed as steps	rootcause.detection_steps and processes/detection_tree_executor.py (execute_step / execute_path, :name parameter substitution).
Column rules with a resolution cache	config.masking_rules + processes/data_masking_engine.py.
Interval scheduling with a live toggle	metrics.registered_processes + the 30-second config watcher.
Calendar scheduling precedent	jobs.jobs / jobs.job_schedules (2,463 rows) and v_job_scheduler.
Run accounting with stage and error code	alerts.mail_send_log and its views, added this week.
Operator page: background run, progress, PDF	/tester and /api/tester/* , added this week.
Bulk load into Postgres	psycpg2 execute_values, already used in utils/transformer_mssql_pg.py.
Packaging a native client driver	utils/oracle_client.py thick-mode initialisation and the spec entries that ship it.